

Ruminations on networking and executors

Credits

Many thanks to Chris Kohlhoff and Niall Douglas for answering my questions about ASIO, the Networking TS, and its work-executing model. Huge thanks to Eric Niebler for answering my thousands of questions about Senders and Receivers with stoic patience.

Massive thanks to Kirk Shoop for correcting my colossal misunderstandings of how the NetTS model works. To some extent, those misunderstandings indicated that the problems would be much worse than they actually are, correcting of which was laudable, considering that Kirk is not a proponent of that design. That shows true class as an engineer, and I can't quite fully express my appreciation for it.

Abstract

This paper makes the plain observation that we should not standardize the Networking TS as it's currently designed. The problem is the use of a P0443 executor. This paper makes the strong recommendation that we shouldn't, in fact, standardize pretty much anything that employs a P0443 executor as a significant component of the design, if it can even be described as a component.

A P0443 executor is not an executor. It's a work-submitter. It doesn't provide any means of finding out whether the work submitted succeeded, or whether an error occurred. Such error handling is fundamental and crucial. Leaving it out from a facility that abstracts the execution of work simply doesn't work as a building block for libraries, larger systems, and applications for an audience as wide as the audience of a C++ standard. The only thing that knows whether the work succeeded is the same facility that accepted the work submission, and trying to entertain the fantasy that the work submission and result observation are separable concepts is just *wrong*.

It certainly gave me no pleasure to write this paper. The original plan about a week earlier than the actual writing of this paper was to write a paper that makes a case for standardizing the Networking TS in C++23, and how that's fine even if we eventually end up with multiple models for executing asynchronous work. The subsequent homework and analysis, however, lead to something completely different. I had high hopes that the Networking TS doesn't actually suffer from the problems of P0443 executors, but that is not the case.

Change history

- R0: Initial version.

The problem

Please read [P2235](#) for some background material for this.

A P0443 executor does not facilitate generic error handling. It doesn't allow writing generic code that can realize that some sort of an error occurred, and it provides no way to report errors that occur after work submission but before invoking a continuation.

This means it doesn't scale. It doesn't work as a component of writing asynchronous code.

The proponents of the Networking TS claim that this is not a problem in the design and model of the TS, or ASIO. But what they fail to see is something that we don't seem to manage to communicate across the disagreeing parties of this matter:

- the Networking TS effectively requires that the executors never fail between work submission and continuation invocation,
- but if we standardize an executor abstraction, that's what programmers will use, beyond their limited use in the Networking model and its APIs.
- C++ programmers will write **thousands** of these executors. Yes, you read that correctly. The users of the Networking TS have maybe dozens of them, on a good day. An asynchronous executor will have orders of magnitude more implementations, many not related to or enjoying the limitations and stricter requirements of networking.

The counter-arguments, and why they don't solve the problem

The proponents of the Networking TS persistently suggest that this is not a problem. They either suggest that programmers can just add error handling support to a refined executor, or that programmers can just compose programs out of completion handlers and the executor is an insignificant part of this bigger picture.

None of that works at all. The completion handlers in the Networking TS provide an associated executor, not a refined executor. There is no API to get a refined executor, or to write code that would work across thousands of different refined executors. The suggested approaches for how to get the error handling to happen in the Networking TS model, or how to compose completion handlers have thus far been very specific small-scale programming solutions that do not scale across a large amount of different implementations of the "concept", because it's not a useful concept that was designed with such programming in mind.

We are going to have thousands of 'executors'. Then we are going to need to solve the same problems all over again, such as simply being able to tell whether the task given to the executor failed, for any reason, including errors that occurred between submission and continuation invocation. No matter of wrapping into completion handlers and asynchronous agents is going to change that; that concept would be the only way to abstract work-runners, and it doesn't provide generic error handling, or generic error detection. That's why all those 'executors' need to be schedulers instead, including the ones that are deep down in the guts of the Networking TS model.

Because they are not deep down, they're not hidden, they're exposed - they are the customization mechanism for users for running continuations for i/o events. The problem of understanding this seems three-fold:

1. An i/o operation can report success/failure to a completion handler, but the executor can fail on its own, and there's no mechanism or channel for reporting that.
2. The task runner that runs a continuation can be arbitrarily complex. From the perspective of the Networking TS, one might think it's just a simple threadpool that never fails to run the continuation, so the complexity is not noticeable for people who write just library wrappers on top of the Networking TS, still doing just networking, and other parts of their application use something else for whatever asynchronous operations. But the executor **is** that "something else", or would be, if it's a scheduler. It can, in addition to just invoking the continuation passed to it, execute an arbitrarily complex preamble or postamble. And many of them will.
3. In order to work with multiple different task runners, we need to perform a from-specific-to-generic translation. The continuation runner is generic, it models a single concept. It can wrap our continuation into a generic continuation. We do not know the specific types of the continuation runner and the

continuation, but we do know the specific i/o operation we want to run. The rest of the exercise is combining these operations, and that doesn't happen by writing more completion handlers, or by somehow relying on a more refined continuation runner or a more refined continuation for the wrapping.

Let's try a pseudo-code example:

```
void run_that_io_operation(scheduler auto sched,
                          sender_of<network_buffer> auto wrapping_continuation)
{
    launch_our_io();
    auto [buffer, status] = fetch_io_results_somewhat(maybe_asynchronously);
    run_on(sched, queue_onto(wrapping_continuation,
                            make_it_queueable(buffer, status)));
}
```

I have no idea what the sched's concrete type is. I have no idea what the wrapping_continuation's concrete type is. They're none of my business, I'm performing a specific i/o operation and passing the results up, for consumption by whatever client. I convert the data I got from my specific i/o operation into a form that can be wrapped in the generic typed wrapper, I perform the wrapping, and then I toss the wrapped continuation over to the scheduler to run.

The continuation wrapper has its own continuation that I don't know about, and it does whatever the calling client wanted to happen when I pass it the i/o result. I don't decide it, the client does. Where that continuation runs, i.e. the scheduler, is not something I decide, the client does.

The client can perform more of these wrappings and queueings and other transformation, doing so in a similar fashion, going from specific-to-generic, using the same generic approach. Its clients can do the same. The scheduler is a Strategy pattern implementation, and the sender is a Decorator pattern implementation, the client can operate on the decorator in a generic fashion, adding decorator layers to it, and all this can be passed to the generic execution strategy without any specific knowledge of what the upper layers concretely did or passed to us, but at the end the upper layers run whatever continuations they needed, translated from our network result.

How many different implementations of this function do I need to support 200 different execution strategies and 10,500 different continuation-decorations(*)?

The 'solutions' suggested by the proponents of the Networking TS model seem to say the answer is "many".

With Senders and Receivers, the answer is "one, you just wrote it."

A P0443 executor can't do this, and it can't interoperate with it. The execution strategy is arbitrarily complex, and the wrapping continuation needs to know if the _strategy_ had an error in invoking the continuation. It also needs to know whether our i/o operation failed, but that's not the only thing that can fail in this whole shebang.

In the Networking TS, the incoming execution strategy would be an executor. That doesn't work, that doesn't scale, that doesn't integrate to the more-generic, which is also higher-abstraction-level and more-complex. It's not just a simple thread-pool that never fails to run the continuation, because the execution strategy is arbitrarily complex and so is the wrapping continuation.

(*)Those numbers of execution strategies and continuation wrappers are not hyperbolic jokes. They are completely arbitrary but they represent different kinds of event loops and task runners that pass asynchronous work results from runners to other runners to event loops to other runners to other event loops, queuing lower-level operation results onto higher-level result handling logic that translates a lower-level result to a higher-level result, ultimately invoking arbitrary application logic.

The algorithms that queue and translate low-level results to higher-level results and low-level errors to high-level results are generic, and the same across layers of different abstraction level. The algorithms that transfer the

results to run as continuations on other runners and other event loops are generic, and the same across layers of different abstraction level. There are adapters that facilitate specific translations, but the algorithms are the same.

Senders and Receivers are designed to be able to do that. A P0443 executor can't work with this. It always requires ignoring the whole pseudo-concept of an executor, and getting at a concrete implementation to fetch the error information, and turning the payload data into something that's composable. The whole concept is completely useless, there's no composed code you can write with it.

On the "need" of executor "guarantees" in the Networking TS

It has been said that Networking TS needs a guarantee that once an execution context for a continuation has been established, that context is used for all invocations of the continuation, whether an error or a success. Two problems:

1. I don't see how the networking library needs this guarantee. Whether a single context is to be used for all invocations of a continuation seems domain-specific and application-specific to me, and nailing it down at the "framework" level seems like a hard-coded choice.
2. A P0443 executor does NOT establish this guarantee. It can decide not to invoke the continuation at all. It can decide to invoke it from different contexts based on what the result of the operation was. It has no such semantics of guaranteeing how the continuation is invoked, all it establishes is an interface for submitting work. Which is not sufficient for establishing semantics for how the work is executed.

With Senders and Receivers, one can apply an algorithm on the chain/nested-pile of work executed to ensure this. And that algorithm can be applied on the higher abstraction layers that know what the actual user needs.

Rephrasing the problem, a summary

Composing P0443 executors is an ad-hoc exercise. It's actually worse than that, because P0443 executors don't compose, you need to compose something more refined and more specific. Composing Networking TS completion handlers or asynchronous operations (which aren't even a thing in the NetTS APIs, so how would I be able to compose those other than in an ad-hoc manner?) seems like an ad-hoc and non-generic exercise. Whether they were designed for or ever deployed in larger-scale compositions that scale from low-level to high-level with any sort of uniformity, I don't know. Such a uniform model could perhaps be built on the completion handlers, but it can't be built on executors, and the problem with that is that everybody will build their own incompatible composition model.

Senders and Receivers were designed to facilitate the generic and scalable composition from the get-go. It's their driving design goal, to separate the work from how it's executed, to translate the results separate from what the work is, to queue both the work and its continuations to different contexts, all concern-separated from the work itself and its continuations, with a uniform model that allows generic algorithms to do the heavy lifting but yet be generic across different abstraction levels and layers. The design recognizes that there are arbitrarily many concrete implementations of a scheduler and a sender, but yet provides common facilities as algorithms that work with all of those concrete implementations, in a composable manner.

On the question of standardising existing practice

"This is not a problem, ASIO has been shipping this for years, and you can find a similar executor in `java.util.concurrent`."

Right. The first one doesn't seem to provide a composition model at all, so maybe these questions haven't come up. Maybe ASIO hasn't been used with an executor that can fail to run the continuation, like a deadline scheduler

that might not be able to lazily acquire the resources needed for running the continuation. Maybe ASIO hasn't been used in an environment where such failures are an issue.

For the java executors, they always have an escape hatch of performing a dynamic cast down to a specific executor that can tell you whether it can report such errors to you. I don't know whether anyone does that, or whether such executors just haven't been used.

Such existing practice has a huge problem that it just plain can't handle errors that occur after submission. That's an irrecoverable data loss, and we know how to do better, because we have other existing practice where this is a solved problem, in libunifex.

Conclusion, i.e. "so what should we do?"

A minimal suggestion

At minimum, we need to axe the P0443 executor from the model of the Networking TS, and replace it with a P2300 scheduler. That way our Networking TS i/o code can fit into the scalable model. There are various impedance mismatches and efficiency concerns about that, so this may be an iffy exercise, and it will take time to design and implement.

A bold suggestion

Here's a rather straightforward suggestion:

1. Stop spending energy on standardizing the Networking TS for C++23.
2. Focus on making progress with Senders and Receivers.

Let's be bold. Let's make a decision. Various analyses lead into this outcome. We have a scalable model, let's later standardize a networking library that fits straight into that scalable model.